Software Requirements Engineering

Presented by: Dr. Abdolraheem Khader

May 2025

Introduction to Requirements Engineering

- Definition: The process of defining, documenting, and maintaining software requirements.
- Objective: To understand what the system should do and the constraints under which it must operate.
- Importance: Ensures that software meets user needs and expectations.

Key Concepts in Requirements Engineering

- Requirements: Statements of what the system must accomplish.
- Stakeholders: Individuals or groups with an interest in the system.
- System Boundaries: Defines the scope of the system.
- Requirements Document: A formal description of system requirements.

Types of Requirements

- Functional Requirements: What the system should do (e.g., data processing).
- Non-Functional Requirements: How the system performs (e.g., performance, usability).
- Domain Requirements: Derived from the application domain (e.g., legal constraints).
- Example: A banking app should handle at least 10,000 transactions per second (Performance).

Requirements Engineering Process (Overview)

- Steps:
  - Elicitation
  - Analysis
  - Specification
  - Validation
  - Management
- (Include a process diagram).

Phase 1 - Requirements Elicitation

- Objective: Gather information from stakeholders.
- Techniques:
  - Interviews.
  - Questionnaires.
  - Observations.
  - Workshops.
- Example: Conducting stakeholder interviews for a healthcare application.

Phase 1 - Elicitation Challenges

- Ambiguous or unclear stakeholder requirements.
- Miscommunication between developers and clients.
- Techniques to Mitigate: Prototyping, structured interviews, and use cases.

Phase 2 - Requirements Analysis

- Purpose: Understand and model requirements.
- Techniques:
  - Data Flow Diagrams (DFD).
  - Use Case Modeling.
  - Scenarios and Storyboards.
- Example: Modeling a login process with a use case diagram.

Requirements Prioritization

- Techniques:
  - MoSCoW (Must have, Should have, Could have, Won't have)
  - Pairwise Comparison
  - Kano Model
- Purpose: Focus on high-impact requirements first.

Requirements Validation

- Goal: Ensure requirements accurately reflect stakeholder needs.
- Methods:
  - Reviews
  - Prototyping
  - Model Checking
  - User Acceptance Testing (UAT)
- Example: Validating a prototype of a user dashboard.

Validation Techniques

- Reviews: Peer review and formal inspection.
- Prototyping: Early visualization of the system.
- Automated Checking: Ensuring formal properties are satisfied.
- Example: Reviewing requirements documents with stakeholders.

Phase 3 - Requirements Specification

- Purpose: Clearly document the system’s functionalities and constraints.
- Format:
  - Textual Description
  - Use Case Diagrams
  - Formal Specifications (e.g., Z Notation)
- Example: Writing functional specifications for a mobile app.

Use Case Modeling

- Definition: Represents user interactions with the system.
- Components:
  - Actors.
  - Use Cases.
  - Relationships
- Example: A use case for online ticket booking.

Scenario-Based Requirements

- Definition: Describing the interaction between users and the system.
- Example: A scenario for online shopping: "User browses, selects a product, and makes a payment.".
- Benefits: Enhances understanding and visualization.

Non-Functional Requirements (NFR)

- Categories:
  - Performance.
  - Security.
  - Usability.
  - Maintainability.
- Challenges: Often harder to quantify and validate.
- Example: The system must respond within 2 seconds.

Managing Changing Requirements

- Sources of Change:
  - Stakeholder feedback
  - Evolving technology
  - Market trends
- Approaches:
  - Version Control
  - Requirement Traceability
  - Change Impact Analysis

Requirements Traceability

- Purpose: Track requirements from origin to implementation.
- Techniques: Traceability matrices and links between requirements and test cases.
- Example: Mapping user stories to test scenarios.

Requirements Management

- Activities:
  - Change Control
  - Versioning
  - Tracking Issues
- Tools:
  - JIRA.
  - DOORS.
  - RequisitePro.

Common Issues in Requirements Engineering

- **Incomplete Requirements:** Missing details lead to project delays.
- **Changing Requirements:** Leads to scope creep.
- **Ambiguity:** Misinterpretation of vague requirements.
- **Mitigation:** Frequent stakeholder engagement and validation.

Best Practices for Requirements Engineering

- **Early Stakeholder Involvement:** Reduces misunderstandings.
- **Clear Documentation:** Avoids ambiguity.
- **Prioritization:** Focuses on critical features.
- **Continuous Validation**: Keeps requirements aligned with needs.

Requirements Engineering in Agile

- **Approach:** Iterative and flexible.
- **User Stories:** Capture requirements as simple, clear statements.
- **Backlog Management:** Continuous prioritization and refinement.
- **Example:** Writing user stories for an e-commerce website.

Case Study: Requirements for a Healthcare System

- **Context:** Developing a patient management system.
- **Elicitation:** Interviews with doctors and administrators.
- **Challenges:** Privacy requirements and data integration.

Industry Tools for Requirements Engineering

- **Collaborative Platforms:** Confluence, JIRA.
- **Modeling Tools:** UML Diagrams, Enterprise Architect.
- **Prototyping:** Axure, Balsamiq.
- **Documentation:** Microsoft Word, Latex.

Real-World Example: Banking Application

- **Functional Requirement:** Transaction processing.
- **Non-Functional Requirement:** Security and data encryption.
- **Challenge:** Integrating with legacy systems.

Metrics for Requirements Quality

- **Completeness:** All aspects covered.
- **Consistency:** No conflicting requirements.
- **Feasibility:** Realistic and achievable.
- **Testability:** Measurable and verifiable.

Workshop Activity

- **Task:** Create a use case diagram for an online learning platform.
- **Discussion:** Identify key functional and non-functional requirements.

Thank You!